

Policy Gradients and RL in the Real World

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية

King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

October 4, 2026

Part	What we cover	Lab
I. Foundations	The agent–environment loop, Markov decision processes, value functions, the Bellman equation, Q-learning	1. Gymnasium basics 2. Q-learning
II. Learning a policy directly	Policy gradients, REINFORCE, baselines and advantages, PPO, GRPO	3. GRPO on Flappy Bird
III. RL in the real world	RLHF and verifiable rewards for language models, multi-agent RL, robotics	—

Part I learns what every action is worth and acts on it.
Part II skips the values and improves the policy itself.

- ▶ **The problem.** An MDP $(\mathcal{S}, \mathcal{A}, R, P, \gamma)$; a policy π maps states to actions; the goal is the largest expected discounted return.
- ▶ **Values.** $V^\pi(s)$: how good a state is. $Q^\pi(s, a)$: how good an action is in that state. The **advantage** $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$: how much better that action is than the policy's average.
- ▶ **Q-learning.** Nudge $Q(s, a)$ toward the Bellman target $r + \gamma \max_{a'} Q(s', a')$, and act ϵ -greedily.

*This afternoon: skip the values and improve the policy itself.
The advantage is how we will decide which actions to make more likely.*

By the end of this session, you will be able to:

- ▶ Derive the **REINFORCE** policy gradient, and explain why baselines and advantages make it learn faster.
- ▶ Explain how **PPO**'s clipped objective keeps each update safe and how **GRPO** replaces the critic with a group baseline, then implement GRPO in Lab 3.
- ▶ Describe where RL is used beyond games (**RLHF** for language models, multi-agent systems, robotics) and what makes each one hard.

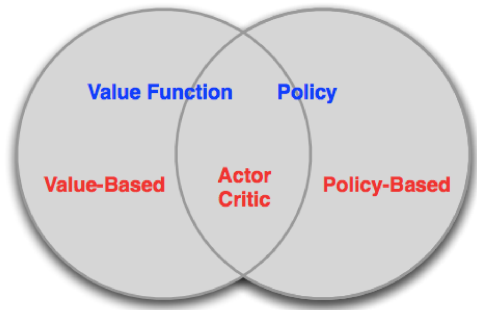
Part II: **Learning a Policy Directly**

Policy gradients, REINFORCE, PPO and GRPO

Lab 3

Reinforcement Learning: **Value-Based** and **Policy-Based** Methods

- ▶ **Value-Based Methods**
 - Learn a value function
 - Derive policy implicitly (e.g., ϵ -greedy)
- ▶ **Policy-Based Methods**
 - Do not learn a value function
 - Learn the policy directly
- ▶ **Actor-Critic Methods** (e.g. *PPO*, later today)
 - Learn both a value function and a policy
 - Combines the strengths of both approaches above



Reinforcement Learning: **Policy Gradients**

- ▶ To act well, you don't need the exact worth of every option — you only need to pick the best one.
- ▶ $Q(s, a)$ answers the **hard** question: *"How much total future reward will I get for every action, in every state?"* — a detailed, complicated function of the whole world.
- ▶ $\pi(s)$ answers the **easy** question: *"What should I do right now?"* — often a simple rule.

- ▶ **To act well, you don't need the exact worth of every option — you only need to pick the best one.**
 - ▶ $Q(s, a)$ answers the **hard** question: *“How much total future reward will I get for every action, in every state?”* — a detailed, complicated function of the whole world.
 - ▶ $\pi(s)$ answers the **easy** question: *“What should I do right now?”* — often a simple rule.
 - ▶ **Grasping:** figuring out the exact long-term payoff of every possible gripper motion is hard. But the actual behavior — “close the hand once it's around the object” — is simple.
- ⇒ So why solve the hard problem (Q) just to act? **Learn the policy directly.**

- Formally, let us define a class of parameterized policies:

$$\Pi = \{\pi_{\theta} \mid \theta \in \mathbb{R}^m\}$$

- For each policy, we can define its expected return:

$$\mathcal{J}(\theta) = \mathbb{E} \left[\sum_{t \geq 0} \gamma^t r_t \mid \pi_{\theta} \right]$$

- Formally, let us define a class of parameterized policies:

$$\Pi = \{\pi_{\theta} \mid \theta \in \mathbb{R}^m\}$$

- For each policy, we can define its expected return:

$$\mathcal{J}(\theta) = \mathbb{E} \left[\sum_{t \geq 0} \gamma^t r_t \mid \pi_{\theta} \right]$$

- Our goal is to find the optimal policy: $\theta^* = \arg \max_{\theta} \mathcal{J}(\theta)$
- How can we achieve this?

- Formally, let us define a class of parameterized policies:

$$\Pi = \{\pi_{\theta} \mid \theta \in \mathbb{R}^m\}$$

- For each policy, we can define its expected return:

$$\mathcal{J}(\theta) = \mathbb{E} \left[\sum_{t \geq 0} \gamma^t r_t \mid \pi_{\theta} \right]$$

- Our goal is to find the optimal policy: $\theta^* = \arg \max_{\theta} \mathcal{J}(\theta)$
- How can we achieve this?
- **Solution:** Perform gradient ascent on the policy parameters!

- **Discrete actions** — Softmax policy:

$$\pi_\theta(a | s) = \frac{e^{\phi(s,a)^\top \theta}}{\sum_{a'} e^{\phi(s,a')^\top \theta}}$$

The network outputs one logit per action; softmax converts them to probabilities.

- **Discrete actions** — Softmax policy:

$$\pi_\theta(a | s) = \frac{e^{\phi(s,a)^\top \theta}}{\sum_{a'} e^{\phi(s,a')^\top \theta}}$$

The network outputs one logit per action; softmax converts them to probabilities.

- **Continuous actions** — Gaussian policy:

$$\pi_\theta(a | s) = \mathcal{N}(\mu_\theta(s), \sigma_\theta(s)^2)$$

The network outputs a mean and standard deviation; actions are sampled from the resulting Gaussian.

- **Discrete actions** — Softmax policy:

$$\pi_\theta(a | s) = \frac{e^{\phi(s,a)^\top \theta}}{\sum_{a'} e^{\phi(s,a')^\top \theta}}$$

The network outputs one logit per action; softmax converts them to probabilities.

- **Continuous actions** — Gaussian policy:

$$\pi_\theta(a | s) = \mathcal{N}(\mu_\theta(s), \sigma_\theta(s)^2)$$

The network outputs a mean and standard deviation; actions are sampled from the resulting Gaussian.

- Gaussian policies are the standard choice for continuous control (e.g. robot joint torques).
- Both are differentiable w.r.t. θ , which is what makes gradient ascent possible.

Reinforcement Learning: **REINFORCE**

- ▶ REINFORCE is an elegant algorithm for maximizing the expected return.
- ▶ Intuition: trial and error.
- ▶ Sample a trajectory τ . If you get a high reward, try to make it more likely; if you get a low reward, try to make it less likely.
- ▶ A trajectory is a sequence of states, actions, and rewards: $\tau = (s_0, a_0, r_0, s_1, a_1, \dots)$.

Our objective is the **expected** discounted return, where $r(\tau) := \sum_{t \geq 0} \gamma^t r_t$ is the return of one trajectory:

$$\mathcal{J}(\theta) = \mathbb{E}_{\tau \sim p(\tau; \theta)} [r(\tau)] = \int_{\tau} r(\tau) p(\tau; \theta) d\tau$$

Reminder: an expectation is a probability-weighted average

$\mathbb{E}_{x \sim p}[f(x)] = \sum_x \underbrace{p(x)}_{\text{how likely}} \underbrace{f(x)}_{\text{value}}$ (or $\int p(x)f(x) dx$ if continuous). Here each “outcome” is a whole trajectory τ : its value is $r(\tau)$ and its weight is how likely the policy is to produce it.

Our objective is the **expected** discounted return, where $r(\tau) := \sum_{t \geq 0} \gamma^t r_t$ is the return of one trajectory:

$$\mathcal{J}(\theta) = \mathbb{E}_{\tau \sim p(\tau; \theta)} [r(\tau)] = \int_{\tau} r(\tau) p(\tau; \theta) d\tau$$

Differentiate w.r.t. θ . The gradient moves inside the integral, and $r(\tau)$ carries no θ (the rewards of a *fixed* trajectory don't change with the weights), so only $p(\tau; \theta)$ is differentiated:

$$\nabla_{\theta} \mathcal{J}(\theta) = \int_{\tau} r(\tau) \nabla_{\theta} p(\tau; \theta) d\tau$$

where the probability of a trajectory factorises as

$$p(\tau; \theta) = \underbrace{p(s_0)}_{\text{start state (no } \theta)} \prod_{t \geq 0} \underbrace{p(s_{t+1} | s_t, a_t)}_{\text{environment (no } \theta)} \underbrace{\pi_{\theta}(a_t | s_t)}_{\text{our policy}}$$

This gradient is **intractable** — for two distinct reasons that the rest of the derivation will fix one at a time.

Problem 1 — we have no way to actually compute this

The integral runs over *every possible trajectory* — far too many to add up directly. Our only tool for such a sum is **Monte Carlo**: run the policy and average what happens. But that works only when each trajectory is weighted by a genuine probability $p(\tau; \theta)$ (non-negative, sums to 1) — then “sampling” means playing episodes. Here the weight is $\nabla_{\theta} p(\tau; \theta)$, a *change* in probability, not a probability: it can be negative and it sums to 0. You cannot play an episode to “sample” a change, so Monte Carlo does not apply.

This gradient is **intractable** — for two distinct reasons that the rest of the derivation will fix one at a time.

Problem 1 — we have no way to actually compute this

The integral runs over *every possible trajectory* — far too many to add up directly. Our only tool for such a sum is **Monte Carlo**: run the policy and average what happens. But that works only when each trajectory is weighted by a genuine probability $p(\tau; \theta)$ (non-negative, sums to 1) — then “sampling” means playing episodes. Here the weight is $\nabla_{\theta} p(\tau; \theta)$, a *change* in probability, not a probability: it can be negative and it sums to 0. You cannot play an episode to “sample” a change, so Monte Carlo does not apply.

Problem 2 — it needs a model of the environment

$p(\tau; \theta)$ contains $p(s_0)$ and $p(s_{t+1}|s_t, a_t)$: the environment's dynamics, which we **do not know** and cannot differentiate.

Goal: rewrite the gradient as an expectation $\mathbb{E}_{\tau \sim p(\tau; \theta)}[\dots]$ — the *only* form we can estimate, by rolling out the policy and averaging.

Goal: rewrite the gradient as an expectation $\mathbb{E}_{\tau \sim p(\tau; \theta)}[\dots]$ — the *only* form we can estimate, by rolling out the policy and averaging.

The log-derivative trick — multiply and divide by $p(\tau; \theta)$:

$$\nabla_{\theta} p(\tau; \theta) = p(\tau; \theta) \frac{\nabla_{\theta} p(\tau; \theta)}{p(\tau; \theta)} = p(\tau; \theta) \nabla_{\theta} \log p(\tau; \theta)$$

This manufactures a factor of $p(\tau; \theta)$ — exactly the sampling weight an expectation needs.

Goal: rewrite the gradient as an expectation $\mathbb{E}_{\tau \sim p(\tau; \theta)}[\dots]$ — the *only* form we can estimate, by rolling out the policy and averaging.

The log-derivative trick — multiply and divide by $p(\tau; \theta)$:

$$\nabla_{\theta} p(\tau; \theta) = p(\tau; \theta) \frac{\nabla_{\theta} p(\tau; \theta)}{p(\tau; \theta)} = p(\tau; \theta) \nabla_{\theta} \log p(\tau; \theta)$$

This manufactures a factor of $p(\tau; \theta)$ — exactly the sampling weight an expectation needs.

Substitute it back into $\nabla_{\theta} \mathcal{J}$:

$$\nabla_{\theta} \mathcal{J}(\theta) = \int_{\tau} (r(\tau) \nabla_{\theta} \log p(\tau; \theta)) p(\tau; \theta) d\tau = \mathbb{E}_{\tau \sim p(\tau; \theta)}[r(\tau) \nabla_{\theta} \log p(\tau; \theta)]$$

It *is* an expectation now $\Rightarrow$ estimate it by Monte Carlo. **Problem 1 solved** (Problem 2 still hides inside $\log p$).

Goal: the log-probability $\log p(\tau; \theta)$ still contains the unknown environment terms. Watch them vanish under ∇_{θ} .

Recall the trajectory probability is a **product** over timesteps:

$$p(\tau; \theta) = p(s_0) \prod_{t \geq 0} p(s_{t+1} | s_t, a_t) \pi_{\theta}(a_t | s_t)$$

Goal: the log-probability $\log p(\tau; \theta)$ still contains the unknown environment terms. Watch them vanish under ∇_{θ} .

Recall the trajectory probability is a **product** over timesteps:

$$p(\tau; \theta) = p(s_0) \prod_{t \geq 0} p(s_{t+1} | s_t, a_t) \pi_{\theta}(a_t | s_t)$$

Taking log turns this product into a **sum** (and since log is increasing, maximizing $\log p$ is the same as maximizing p):

$$\log p(\tau; \theta) = \log p(s_0) + \sum_{t \geq 0} \left[\log p(s_{t+1} | s_t, a_t) + \log \pi_{\theta}(a_t | s_t) \right]$$

Goal: the log-probability $\log p(\tau; \theta)$ still contains the unknown environment terms. Watch them vanish under ∇_{θ} .

Recall the trajectory probability is a **product** over timesteps:

$$p(\tau; \theta) = p(s_0) \prod_{t \geq 0} p(s_{t+1} | s_t, a_t) \pi_{\theta}(a_t | s_t)$$

Taking log turns this product into a **sum** (and since log is increasing, maximizing $\log p$ is the same as maximizing p):

$$\log p(\tau; \theta) = \log p(s_0) + \sum_{t \geq 0} \left[\log p(s_{t+1} | s_t, a_t) + \log \pi_{\theta}(a_t | s_t) \right]$$

Differentiate. Every term without θ becomes zero — the start state $\log p(s_0)$ and *all* dynamics $\log p(s_{t+1} | s_t, a_t)$:

$$\nabla_{\theta} \log p(\tau; \theta) = \sum_{t \geq 0} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

REINFORCE: Fixing Problem 2 — The Dynamics Drop Out

Goal: the log-probability $\log p(\tau; \theta)$ still contains the unknown environment terms. Watch them vanish under ∇_{θ} .

Recall the trajectory probability is a **product** over timesteps:

$$p(\tau; \theta) = p(s_0) \prod_{t \geq 0} p(s_{t+1} | s_t, a_t) \pi_{\theta}(a_t | s_t)$$

Taking log turns this product into a **sum** (and since log is increasing, maximizing $\log p$ is the same as maximizing p):

$$\log p(\tau; \theta) = \log p(s_0) + \sum_{t \geq 0} \left[\log p(s_{t+1} | s_t, a_t) + \log \pi_{\theta}(a_t | s_t) \right]$$

Differentiate. Every term without θ becomes zero — the start state $\log p(s_0)$ and *all* dynamics $\log p(s_{t+1} | s_t, a_t)$:

$$\nabla_{\theta} \log p(\tau; \theta) = \sum_{t \geq 0} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

Only our own policy survives $\Rightarrow$ the estimator is **model-free**. **Problem 2 solved.**

Put both fixes together: start from the expectation (Problem 1 fix) and substitute $\nabla_{\theta} \log p = \sum_{t \geq 0} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$ (Problem 2 fix):

$$\begin{aligned}\nabla_{\theta} \mathcal{J}(\theta) &= \mathbb{E}_{\tau} [r(\tau) \nabla_{\theta} \log p(\tau; \theta)] \\ &= \mathbb{E}_{\tau} \left[r(\tau) \sum_{t \geq 0} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right] \approx \sum_{t \geq 0} r(\tau) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)\end{aligned}$$

(Last step: one sampled trajectory τ . Here $r(\tau)$ is constant over t , so it could equally sit outside the sum — we keep it inside because the next section replaces it with a *per-timestep* weight.)

Put both fixes together: start from the expectation (Problem 1 fix) and substitute $\nabla_{\theta} \log p = \sum_{t \geq 0} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$ (Problem 2 fix):

$$\begin{aligned}\nabla_{\theta} \mathcal{J}(\theta) &= \mathbb{E}_{\tau} [r(\tau) \nabla_{\theta} \log p(\tau; \theta)] \\ &= \mathbb{E}_{\tau} \left[r(\tau) \sum_{t \geq 0} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right] \approx \sum_{t \geq 0} r(\tau) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)\end{aligned}$$

(Last step: one sampled trajectory τ . Here $r(\tau)$ is constant over t , so it could equally sit outside the sum — we keep it inside because the next section replaces it with a *per-timestep* weight.)

Interpretation: $\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$ points in the direction that makes a_t *more likely*; the scalar $r(\tau)$ signs and scales that push:

- ▶ $r(\tau)$ high $\Rightarrow$ increase the probability of the actions taken;
- ▶ $r(\tau)$ low / negative $\Rightarrow$ decrease them.

Put both fixes together: start from the expectation (Problem 1 fix) and substitute $\nabla_{\theta} \log p = \sum_{t \geq 0} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$ (Problem 2 fix):

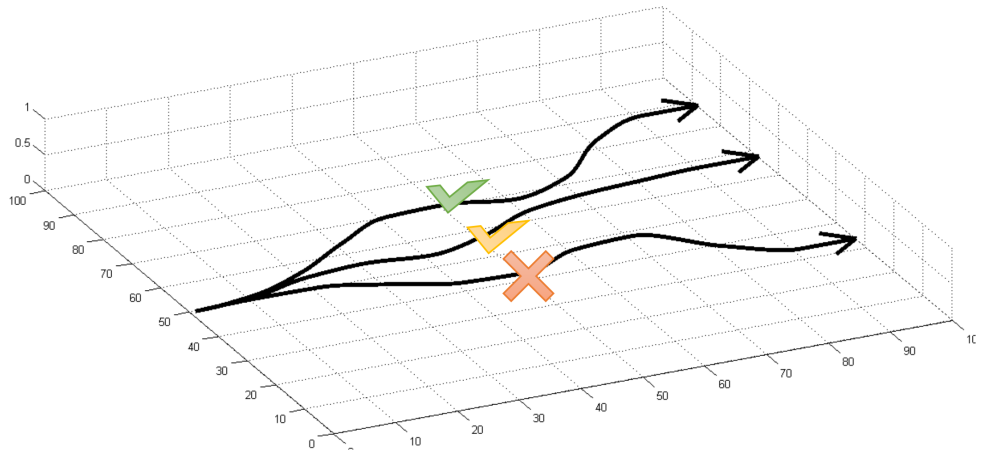
$$\begin{aligned}\nabla_{\theta} \mathcal{J}(\theta) &= \mathbb{E}_{\tau} [r(\tau) \nabla_{\theta} \log p(\tau; \theta)] \\ &= \mathbb{E}_{\tau} \left[r(\tau) \sum_{t \geq 0} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right] \approx \sum_{t \geq 0} r(\tau) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)\end{aligned}$$

(Last step: one sampled trajectory τ . Here $r(\tau)$ is constant over t , so it could equally sit outside the sum — we keep it inside because the next section replaces it with a *per-timestep* weight.)

Interpretation: $\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$ points in the direction that makes a_t *more likely*; the scalar $r(\tau)$ signs and scales that push:

- ▶ $r(\tau)$ high $\Rightarrow$ increase the probability of the actions taken;
- ▶ $r(\tau)$ low / negative $\Rightarrow$ decrease them.

Crediting every action with the *whole*-trajectory return looks crude — but it is **unbiased**: in expectation, it averages out.



Reinforcement Learning: **Variance Reduction**

Our gradient is estimated from *sampled* trajectories, so it is never exact. There are two independent ways such an estimate can be wrong:

Variance — how much it bounces

Run the same policy again and you get different trajectories, hence a different gradient estimate. **High variance** = the estimate jumps around from one batch to the next $\Rightarrow$ you need many samples to average out the noise $\Rightarrow$ slow, unstable learning.

Bias — whether it points the right way on average

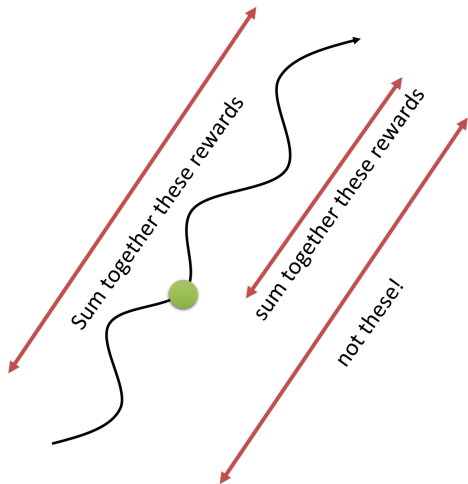
Unbiased = averaged over infinitely many samples it equals the true gradient $\nabla_{\theta} \mathcal{J}$. **Biased** = systematically off, converging toward a slightly wrong direction *even with infinite data*.

Dartboard picture: **variance** is how scattered the darts land; **bias** is whether they cluster off the bullseye. *The whole game below*: accept a little bias if it buys a large drop in variance — the net result learns faster.

- ▶ However, there is a problem.
- ▶ This approach suffers from high variance because credit assignment is difficult.
- ▶ Can we help the estimator?

- ▶ However, there is a problem.
- ▶ This approach suffers from high variance because credit assignment is difficult.
- ▶ Can we help the estimator?
- ▶ **First idea — the Causality Trick:** Rewards before time t cannot be affected by action a_t , so they carry no useful gradient signal. Use only future rewards:

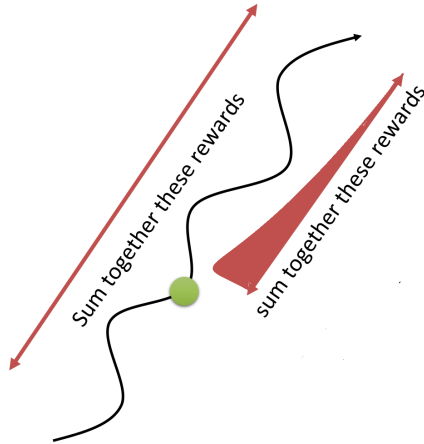
$$\nabla_{\theta} \mathcal{J}(\theta) \approx \sum_{t \geq 0} \left(\sum_{t' \geq t} r_{t'} \right) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$



- ▶ The causality trick is **unbiased** — removing past rewards is mathematically correct, so on average the gradient is unchanged.
- ▶ But the reward-to-go $\sum_{t' \geq t} r_{t'}$ is still a sum over an *entire* future rollout, driven by many random action choices and random environment transitions. Run the **same** policy twice and this number can come out wildly different.
- ▶ That swing is the **high variance**: the gradient estimate bounces from sample to sample, so it takes a huge number of trajectories to average out — learning is slow and unstable.

- ▶ The causality trick is **unbiased** — removing past rewards is mathematically correct, so on average the gradient is unchanged.
- ▶ But the reward-to-go $\sum_{t' \geq t} r_{t'}$ is still a sum over an *entire* future rollout, driven by many random action choices and random environment transitions. Run the **same** policy twice and this number can come out wildly different.
- ▶ That swing is the **high variance**: the gradient estimate bounces from sample to sample, so it takes a huge number of trajectories to average out — learning is slow and unstable.
- ▶ **Second idea — discounting**: weight future rewards by $\gamma^{t'-t}$, so distant, largely-unrelated rewards count for less. This trades a small **bias** for a large drop in variance:

$$\nabla_{\theta} \mathcal{J}(\theta) \approx \sum_{t \geq 0} \left(\sum_{t' \geq t} \gamma^{t'-t} r_{t'} \right) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$



- **Problem:** The raw value of a trajectory isn't necessarily meaningful. For example, if all rewards are positive, you keep increasing the probabilities of actions.

- ▶ **Problem:** The raw value of a trajectory isn't necessarily meaningful. For example, if all rewards are positive, you keep increasing the probabilities of actions.
- ▶ **What is important then?** Whether a reward is better or worse than what you expect to get.

- ▶ **Problem:** The raw value of a trajectory isn't necessarily meaningful. For example, if all rewards are positive, you keep increasing the probabilities of actions.
- ▶ **What is important then?** Whether a reward is better or worse than what you expect to get.
- ▶ **Idea:** Introduce a baseline function dependent on the state:

$$\nabla_{\theta} \mathcal{J}(\theta) \approx \sum_{t \geq 0} \left(\sum_{t' \geq t} \gamma^{t'-t} r_{t'} - b(s_t) \right) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

- ▶ **Problem:** The raw value of a trajectory isn't necessarily meaningful. For example, if all rewards are positive, you keep increasing the probabilities of actions.
- ▶ **What is important then?** Whether a reward is better or worse than what you expect to get.
- ▶ **Idea:** Introduce a baseline function dependent on the state:

$$\nabla_{\theta} \mathcal{J}(\theta) \approx \sum_{t \geq 0} \left(\sum_{t' \geq t} \gamma^{t'-t} r_{t'} - b(s_t) \right) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

- ▶ A simple baseline: the moving average of rewards experienced so far from all trajectories.

- ▶ Can we choose a better baseline?
- ▶ Essentially, we want to increase the probability of an action from a state if this action was better than the **expected value** from that state.

- ▶ Can we choose a better baseline?
- ▶ Essentially, we want to increase the probability of an action from a state if this action was better than the **expected value** from that state.
- ▶ What does this remind you of?

- ▶ Can we choose a better baseline?
- ▶ Essentially, we want to increase the probability of an action from a state if this action was better than the **expected value** from that state.
- ▶ What does this remind you of?
- ▶ **Recall from RL Foundations:** We defined the **Advantage function**:

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$$

- ▶ $A^{\pi}(s_t, a_t) > 0$: action a_t was better than average — increase its probability.
- ▶ $A^{\pi}(s_t, a_t) < 0$: action a_t was worse than average — decrease its probability.

- ▶ Using the advantage as our baseline gives us the estimator:

$$\nabla_{\theta} \mathcal{J}(\theta) \approx \sum_{t \geq 0} A^{\pi}(s_t, a_t) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

- ▶ This is the best possible baseline: it directly measures how much better an action was than the policy's own average.
- ▶ It is unbiased, and it provides lower variance than a fixed moving-average baseline.

*But how do we actually estimate A^{π} ? That — learning a **critic** — is exactly what Actor-Critic methods are about.*

Initialize policy network π_θ with random weights θ ; set step size α and discount γ

for $episode = 1$ **to** M **do**

 Run π_θ for one full episode of length T , producing

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_{T-1}, a_{T-1}, r_{T-1})$$

for $t = 0$ **to** $T - 1$ **do**

 | Reward-to-go from step t : $R_t = \sum_{t'=t}^{T-1} \gamma^{t'-t} r_{t'}$

end

 Gradient *ascent* update (reward-to-go minus baseline $b(s_t)$):

$$\theta \leftarrow \theta + \alpha \sum_{t=0}^{T-1} (R_t - b(s_t)) \nabla_\theta \log \pi_\theta(a_t | s_t)$$

end

Algorithm: REINFORCE (Monte-Carlo Policy Gradient)

⁰Williams (1992), *Simple statistical gradient-following algorithms for connectionist reinforcement learning*, *Machine Learning* 8. An episode of length T has actions $a_0, \dots, a_{T-1}$ (T in total); the terminal state s_T has no action, so the sum stops at $T - 1$. The baseline $b(s_t)$ is optional — $b \equiv 0$ gives vanilla REINFORCE, and a running average of returns is a simple choice. We *ascend* because we maximize expected return $\mathcal{J}(\theta)$, whereas Q-learning minimized the Bellman error (descent).

- ▶ Policy gradient and Q-learning use two fundamentally different representations: policies and value functions.
- ▶ Advantage of both methods: no need to model the environment.

- ▶ Policy gradient and Q-learning use two fundamentally different representations: policies and value functions.
- ▶ Advantage of both methods: no need to model the environment.
- ▶ **Policy Gradient: Pros and Cons**
 - **Pros:** Unbiased estimate of the gradient of expected return.
 - Can handle large action spaces (since only one action needs to be sampled).
 - **Cons:** High variance updates (leads to poor sample efficiency).
 - Does not perform credit assignment effectively.

- ▶ Policy gradient and Q-learning use two fundamentally different representations: policies and value functions.
- ▶ Advantage of both methods: no need to model the environment.
- ▶ **Policy Gradient: Pros and Cons**
 - **Pros:** Unbiased estimate of the gradient of expected return.
 - Can handle large action spaces (since only one action needs to be sampled).
 - **Cons:** High variance updates (leads to poor sample efficiency).
 - Does not perform credit assignment effectively.
- ▶ **Q-Learning: Pros and Cons**
 - **Pros:** Lower variance updates, more sample efficient.
 - Performs credit assignment.
 - **Cons:** Biased updates due to function approximation.
 - Difficult to handle large action spaces (since the maximum over actions must be computed).

Where Does $\hat{A}_t$ Come From? Three Answers

Every method from here on takes the same step, $\theta \leftarrow \theta + \alpha \sum_t \hat{A}_t \nabla_{\theta} \log \pi_{\theta}(a_t|s_t)$. They differ only in how they estimate the advantage $\hat{A}_t$:

REINFORCE: a sampled return minus a baseline

$\hat{A}_t = \sum_{t' \geq t} \gamma^{t'-t} r_{t'} - b(s_t)$. Unbiased, but it rests on a single noisy rollout.

Where Does $\hat{A}_t$ Come From? Three Answers

Every method from here on takes the same step, $\theta \leftarrow \theta + \alpha \sum_t \hat{A}_t \nabla_{\theta} \log \pi_{\theta}(a_t|s_t)$. They differ only in how they estimate the advantage $\hat{A}_t$:

REINFORCE: a sampled return minus a baseline

$\hat{A}_t = \sum_{t' \geq t} \gamma^{t'-t} r_{t'} - b(s_t)$. Unbiased, but it rests on a single noisy rollout.

Actor-critic (PPO): learn a second network, the critic

A **critic** $V_{\phi}(s)$ is trained to predict returns. One real reward plus the critic's guess for the rest gives $\hat{A}_t \approx r_t + \gamma V_{\phi}(s_{t+1}) - V_{\phi}(s_t)$. Lower variance, but now two networks train together.

Where Does $\hat{A}_t$ Come From? Three Answers

Every method from here on takes the same step, $\theta \leftarrow \theta + \alpha \sum_t \hat{A}_t \nabla_{\theta} \log \pi_{\theta}(a_t|s_t)$. They differ only in how they estimate the advantage $\hat{A}_t$:

REINFORCE: a sampled return minus a baseline

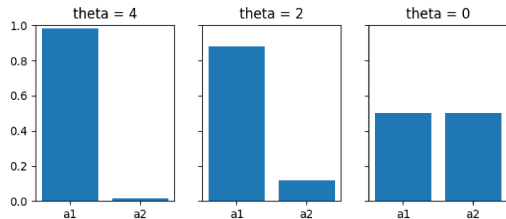
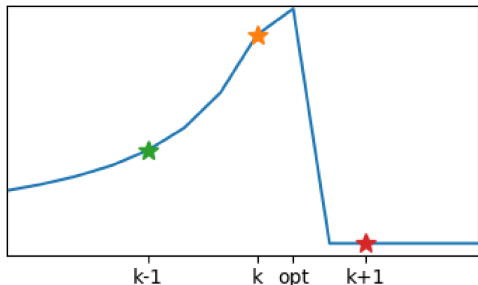
$\hat{A}_t = \sum_{t' \geq t} \gamma^{t'-t} r_{t'} - b(s_t)$. Unbiased, but it rests on a single noisy rollout.

Actor-critic (PPO): learn a second network, the critic

A **critic** $V_{\phi}(s)$ is trained to predict returns. One real reward plus the critic's guess for the rest gives $\hat{A}_t \approx r_t + \gamma V_{\phi}(s_{t+1}) - V_{\phi}(s_t)$. Lower variance, but now two networks train together.

Group baseline (GRPO): compare attempts at the same task

Make G attempts from the same start (or answer the same prompt G times) and score each against the group: $\hat{A}_i = R_i - \text{mean}(R_1, \dots, R_G)$. No critic at all.



- ▶ A large gradient step in *parameter space* can cause a catastrophically large shift in *policy space*
- ▶ Once a policy collapses (e.g., always picks one bad action), the agent collects bad data and the policy is hard to recover
- ▶ We need to control how much the policy distribution changes, not just how much θ changes

- ▶ Last slide's conclusion: *don't let the policy move too far in one update*. But to act on that we first need to answer a prior question:

Given a candidate π_θ , how much better or worse is it than $\pi_{\theta_{\text{old}}}$ — without running it?

- ▶ Why we can't just run it: evaluating $\mathcal{J}(\theta)$ means collecting a fresh batch of rollouts for every candidate θ we consider. That is prohibitively expensive — and if the candidate is bad, we only find out by damaging the agent.
- ⇒ So we need a **score we can compute from the batch we already collected** — a stand-in for $\mathcal{J}(\theta)$, called a **surrogate objective**. Next slide builds it from scratch.

What the batch contains: for every timestep, the state s_t , the action a_t we took, and $\hat{A}_t$ — whether that action turned out better (+) or worse (−) than usual.

- ▶ A candidate π_θ cannot change what happened. The *only* thing it changes is **how likely each of those actions would have been**.

¹ $r_t(\theta)$ is the probability ratio, not the reward r_t from Part I — distinguished by the θ argument.

What the batch contains: for every timestep, the state s_t , the action a_t we took, and $\hat{A}_t$ — whether that action turned out better (+) or worse (−) than usual.

- ▶ A candidate π_θ cannot change what happened. The *only* thing it changes is **how likely each of those actions would have been**.
- ▶ So the question collapses to: *does π_θ put more probability on the actions that went well, and less on the ones that went badly?*

¹ $r_t(\theta)$ is the probability ratio, not the reward r_t from Part I — distinguished by the θ argument.

What the batch contains: for every timestep, the state s_t , the action a_t we took, and $\hat{A}_t$ — whether that action turned out better (+) or worse (−) than usual.

- ▶ A candidate π_θ cannot change what happened. The *only* thing it changes is **how likely each of those actions would have been**.
- ▶ So the question collapses to: *does π_θ put more probability on the actions that went well, and less on the ones that went badly?*
- ▶ Measure that per action with the **probability ratio**:¹

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

1 : would take a_t just as often
2 : twice as often 0.5 : half as often

¹ $r_t(\theta)$ is the probability ratio, not the reward r_t from Part I — distinguished by the θ argument.

What the batch contains: for every timestep, the state s_t , the action a_t we took, and $\hat{A}_t$ — whether that action turned out better (+) or worse (−) than usual.

- ▶ A candidate π_θ cannot change what happened. The *only* thing it changes is **how likely each of those actions would have been**.
- ▶ So the question collapses to: *does π_θ put more probability on the actions that went well, and less on the ones that went badly?*
- ▶ Measure that per action with the **probability ratio**:¹

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

1 : would take a_t just as often
2 : twice as often 0.5 : half as often

- ▶ Score each sample by **(how much more the new policy wants it) × (how good it was)**, and average:

$$\mathcal{L}(\theta) = \mathbb{E}_t \left[r_t(\theta) \hat{A}_t \right]$$

¹ $r_t(\theta)$ is the probability ratio, not the reward r_t from Part I — distinguished by the θ argument.

Every column is a number we can just compute: $\hat{A}_t$ from the critic, the probabilities by evaluating each network on (s_t, a_t) .

	$\hat{A}_t$	$\pi_{\theta_{\text{old}}}(a_t s_t)$	$\pi_{\theta}(a_t s_t)$	$r_t(\theta)$	$r_t(\theta) \hat{A}_t$
went well	+2.0	0.20	0.40	2.0	+4.0
went badly	-1.0	0.50	0.40	0.8	-0.8

- $\mathcal{L}(\theta) = \frac{1}{2}(4.0 - 0.8) = +1.6 > 0$: the candidate **doubles down on what worked, backs off what didn't**. Ascending $\mathcal{L}$ searches for exactly that.

Every column is a number we can just compute: $\hat{A}_t$ from the critic, the probabilities by evaluating each network on (s_t, a_t) .

	$\hat{A}_t$	$\pi_{\theta_{\text{old}}}(a_t s_t)$	$\pi_{\theta}(a_t s_t)$	$r_t(\theta)$	$r_t(\theta) \hat{A}_t$
went well	+2.0	0.20	0.40	2.0	+4.0
went badly	-1.0	0.50	0.40	0.8	-0.8

- ▶ $\mathcal{L}(\theta) = \frac{1}{2}(4.0 - 0.8) = +1.6 > 0$: the candidate **doubles down on what worked, backs off what didn't**. Ascending $\mathcal{L}$ searches for exactly that.
- ▶ **Why the ratio is the right multiplier**: the batch was collected by the *old* policy, so each action appears about as often as *it* would have chosen it. If the new policy would pick that action twice as often, the sample should count twice. That re-counting is all the ratio does.

Every column is a number we can just compute: $\hat{A}_t$ from the critic, the probabilities by evaluating each network on (s_t, a_t) .

	$\hat{A}_t$	$\pi_{\theta_{\text{old}}}(a_t s_t)$	$\pi_{\theta}(a_t s_t)$	$r_t(\theta)$	$r_t(\theta) \hat{A}_t$
went well	+2.0	0.20	0.40	2.0	+4.0
went badly	-1.0	0.50	0.40	0.8	-0.8

- ▶ $\mathcal{L}(\theta) = \frac{1}{2}(4.0 - 0.8) = +1.6 > 0$: the candidate **doubles down on what worked, backs off what didn't**. Ascending $\mathcal{L}$ searches for exactly that.
- ▶ **Why the ratio is the right multiplier**: the batch was collected by the *old* policy, so each action appears about as often as *it* would have chosen it. If the new policy would pick that action twice as often, the sample should count twice. That re-counting is all the ratio does.
- ▶ At $\theta = \theta_{\text{old}}$ every $r_t = 1$, so $\mathcal{L}$ starts at ≈ 0 and its gradient is exactly the vanilla policy gradient. **This is the objective PPO maximizes.**

$\mathcal{L}(\theta)$ is an *approximation* of the true improvement, and it degrades in two ways as π_θ moves away from $\pi_{\theta_{\text{old}}}$:

1. The re-weighting becomes statistically useless

If $\pi_{\theta_{\text{old}}}(a|s)$ is tiny where $\pi_\theta(a|s)$ is large, $r_t(\theta)$ blows up: one rare sample gets an enormous weight and swings the entire gradient. The estimate stays unbiased but its **variance explodes**.

2. We quietly swapped the state distribution

Importance sampling corrects for the *actions*, but we still evaluate them at the states $\pi_{\theta_{\text{old}}}$ happened to visit. A very different policy visits very different states — so the surrogate scores the new policy on a world it will no longer see.

- ▶ Both errors are **small when the two policies are close**, and unbounded when they are not.
- ⇒ This is the principled version of “big steps break things”. The fix is one idea: **maximize $\mathcal{L}(\theta)$ while keeping π_θ close to $\pi_{\theta_{\text{old}}}$** . PPO is the standard way to enforce “close”.

Policy Optimization: **PPO**

- ▶ The direct way to keep π_θ close is a hard limit on how far it may move per update, a **trust region**. TRPO (Schulman et al., 2015) enforces it exactly, but needs second-order optimisation (conjugate gradient, Fisher matrix) — expensive and complex to implement
- ▶ **Question:** can we get the same stability with plain SGD?

- ▶ The direct way to keep π_θ close is a hard limit on how far it may move per update, a **trust region**. TRPO (Schulman et al., 2015) enforces it exactly, but needs second-order optimisation (conjugate gradient, Fisher matrix) — expensive and complex to implement
- ▶ **Question:** can we get the same stability with plain SGD?
- ▶ **PPO** (Schulman et al., 2017) proposes two variants:
 - **PPO-KL:** add a KL penalty to the objective and adapt the penalty coefficient β over training
 - **PPO-Clip:** clip the probability ratio $r_t(\theta)$ to prevent large policy updates
- ▶ PPO-Clip is the standard variant — simpler, no hyperparameter tuning on β , works at least as well empirically

► Recall the probability ratio: $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$

► The **PPO-Clip objective** is:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right]$$

► ε is a small hyperparameter (typically $\varepsilon = 0.2$)

► Recall the probability ratio: $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$

► The **PPO-Clip objective** is:

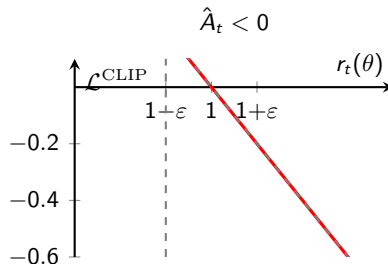
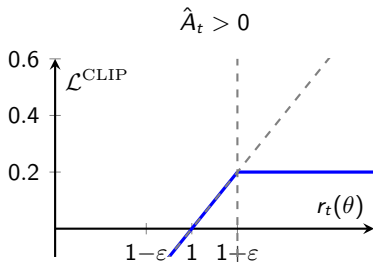
$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right]$$

► ε is a small hyperparameter (typically $\varepsilon = 0.2$)

► **Why clip + min?**

- When $\hat{A}_t > 0$: clipping caps r_t at $1 + \varepsilon$, removing the incentive to increase the ratio further
- When $\hat{A}_t < 0$: clipping floors r_t at $1 - \varepsilon$, removing the incentive to decrease the ratio further
- The **min** ensures the objective is a lower bound — we never benefit from going outside $[1 - \varepsilon, 1 + \varepsilon]$

► This enforces a soft trust region without any second-order computation



- ▶ **Left ($\hat{A}_t > 0$):** benefit grows linearly with r_t until $1 + \epsilon$, then is clipped flat — no reward for pushing the policy further
- ▶ **Right ($\hat{A}_t < 0$):** penalty grows as r_t decreases until $1 - \epsilon$, then is clipped flat
- ▶ Gray dashed line = unclipped surrogate; colored solid = clipped (what PPO optimises)

Policy Optimization: **GRPO**

- ▶ The fragile part of the Actor-Critic loop is the **critic** V_ϕ — training two networks together is the main source of instability
- ▶ **Group Relative Policy Optimization** (GRPO; Shao et al., 2024) removes the critic entirely

- ▶ The fragile part of the Actor-Critic loop is the **critic** V_ϕ — training two networks together is the main source of instability
- ▶ **Group Relative Policy Optimization** (GRPO; Shao et al., 2024) removes the critic entirely
- ▶ Per prompt/state, sample a **group** of G outputs under $\pi_{\theta_{\text{old}}}$, scored with rewards R_i
- ▶ Use the **group mean as the baseline** — a Monte Carlo advantage, no value function:

$$\hat{A}_i = \frac{R_i - \overbrace{\text{mean}(R_1, \dots, R_G)}^{\text{the "Group Relative" baseline}}}{\underbrace{\text{std}(R_1, \dots, R_G)}_{\text{per-group advantage normalisation}}}$$

- ▶ Only the **mean-subtraction** is the GRPO contribution — that is the “Group Relative” baseline replacing the critic V_ϕ
- ▶ The **std-normalisation** is a *separate* stabilisation trick: it is the same per-minibatch advantage normalisation standard PPO implementations use, applied per group — not part of the baseline
- ▶ So a Monte Carlo baseline does *not* “need” variance correction; the std term is an orthogonal, optional stabiliser

- ▶ Only the **mean-subtraction** is the GRPO contribution — that is the “Group Relative” baseline replacing the critic V_ϕ
- ▶ The **std-normalisation** is a *separate* stabilisation trick: it is the same per-minibatch advantage normalisation standard PPO implementations use, applied per group — not part of the baseline
- ▶ So a Monte Carlo baseline does *not* “need” variance correction; the std term is an orthogonal, optional stabiliser
- ▶ Plug $\hat{A}_i$ into the *same* PPO-Clip objective (plus a KL term to a reference policy)
- ▶ **Why it matters:** no critic $\Rightarrow$ no two-network instability and $\approx$ half the memory
- ▶ Since 2024 GRPO is the default for **LLM RL** (DeepSeek-R1, many open recipes) — next, in Part III: **RLHF**, where the reward is a learned preference model

One update: play a **group** of $G = 8$ episodes with the current policy, score each episode by its return R_i , then take a few PPO-Clip gradient steps on that data.

- ▶ **TASK 1, the group advantage:** $\hat{A}_i = \frac{R_i - \text{mean}(R)}{\text{std}(R)}$, shared by every step of episode i .
- ▶ **TASK 2, the probability ratio:** $r_t(\theta) = \exp(\log \pi_\theta(a_t|s_t) - \log \pi_{\theta_{\text{old}}}(a_t|s_t))$.
- ▶ **TASK 3, the clipped surrogate:** $\text{loss} = -\text{mean}[\min(r_t \hat{A}, \text{clip}(r_t, 1 - \varepsilon, 1 + \varepsilon) \hat{A})]$, minus a small entropy bonus that keeps the policy exploring.

What to expect

The bird learns to flap and clears the first pipe or two, not a high score. One number per episode is coarse credit assignment: every flap in a run gets the same advantage, which is the variance problem from the REINFORCE section. A critic (PPO) or per-step advantages would learn faster.

Part III: **RL** in the Real World

RLHF, multi-agent RL and robotics

- ▶ **RLHF (the focus).** When the reward is hard to write down, *learn* it from human (or AI, or verifiable) feedback. This is how modern LLMs are aligned, and it runs on the PPO and GRPO from Part II.
- ▶ **Multi-Agent RL (overview).** What changes when other learning agents share your environment and the ground keeps shifting underneath you.
- ▶ **Robotics (overview).** What breaks when the environment is physical: sample cost, safety, resets, and the sim-to-real gap.

Plus one slide on RL running computer systems.

- ▶ Every method so far assumed a reward function r we could *write down*: the game score, −distance to a target, +1 for a win.
- ▶ Now suppose the goal is “*be a helpful, honest, harmless assistant.*” What number measures that? There is no formula.

Answer A. “It’s the force that pulls things down. It’s why a ball falls when you let go of it.”

Answer B. “Gravity is the curvature of space-time sourced by the stress–energy tensor, $G_{\mu\nu} = 8\pi T_{\mu\nu} \dots$ ”

Prompt: “Explain gravity to a 6-year-old.”

Almost everyone prefers **A**, yet *no one can write the formula that says so.*

- ▶ *When you cannot write the reward, recover it from data.* This is called *inverse RL*, and RLHF is its most consequential example.
- ▶ People are bad at giving a *number* (“this answer is a 7.3”) but good at *comparing*: “A is better than B.” Comparisons are cheap and consistent.
- ▶ **The plan, in three moves:**
 1. Collect many human *comparisons* of model answers.
 2. Train a model to predict those comparisons; this learned model *becomes our reward signal*.
 3. Use RL to push the policy toward answers that score well under it.
- ▶ This is **Reinforcement Learning from Human Feedback (RLHF)**, the method behind ChatGPT-style assistants.

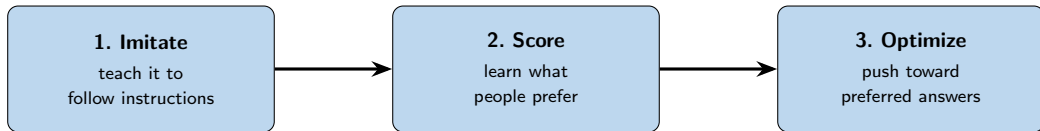
Reinforcement Learning **from** Human Feedback for Language Models

Everything that follows is just the familiar RL language, applied to text.

- ▶ **Prompt** x , the input (a question or instruction). *the state*
- ▶ **Response** y , the model's full answer. *the action*
- ▶ **Policy** $\pi_{\theta}(y \mid x)$, the language model itself: given a prompt, it assigns a probability to possible responses and samples one. The weights θ are what we train.
- ▶ It writes the answer one *token* (word-piece) at a time; at each step a final layer, the **LM head**, outputs a probability over the next token.

So “train the policy” simply means “adjust the language model’s weights θ .”

Three stages. We build each one up over the next slides; for now, just the plain English.



- ▶ **Start:** a raw language model that can produce text but doesn't reliably do what you ask.
- ▶ **End:** a model that follows instructions *and* leans toward the answers people actually want.

- ▶ **Problem:** a raw pretrained model can *continue* text, but doesn't reliably answer questions or follow instructions.
- ▶ **Fix: show it examples.** Collect prompts paired with *human-written ideal answers*, and train the model to imitate them. This is ordinary supervised learning (next-token prediction), no RL yet.
- ▶ The result is a decent instruction-follower. We **freeze a copy of it** and call it the *reference model* π_{ref} ; it will be our anchor in Stages 2 and 3.

SFT can only *copy* the demonstrations it was given. To go *beyond* them, we need to capture what people *prefer*. That is Stage 2.

- ▶ We want a **reward model** r_ϕ : a network (its own weights ϕ) that reads a (prompt, response) and outputs *a single number (how good is this answer?)*. Higher means better.
- ▶ **Where does its training data come from?** Comparisons. For one prompt x , show a human two candidate responses; they pick the better one. We label the pair:
 - y^+ , the response the human **preferred**,
 - y^- , the response they **rejected**.
- ▶ **What we want from r_ϕ :** give y^+ a higher score than y^- , on every pair. Next slide: how to turn “preferred” into a trainable loss.

How likely is a person to prefer y^+ over y^- ? It should grow with the **gap in their scores**, squeezed into a probability by the sigmoid σ (" $\succ$ " reads "*is preferred over*"):

$$\underbrace{P(y^+ \succ y^- \mid x)}_{\text{chance the human picks } y^+} = \sigma(r(y^+) - r(y^-)), \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

Train r_ϕ to make these probabilities match the human labels (maximum likelihood). The negative log-likelihood is the **reward-model loss**:

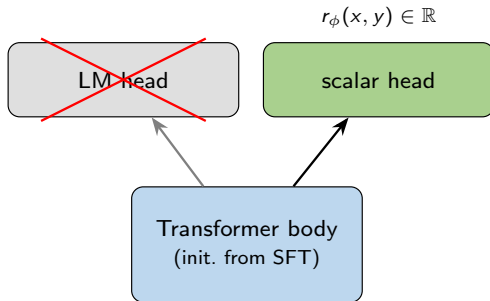
$$L_{\text{RM}} = -\log \sigma(r_\phi(y^+) - r_\phi(y^-))$$

- ▶ In plain words: *push the preferred answer's score above the rejected one's*.
- ▶ Remember this $-\log \sigma(\Delta)$ shape: the **exact same form** comes back when we reach DPO.
- ▶ Only the *gap* between scores matters; the absolute scale is free.

What Is the Reward Model, Concretely?

We've said *what* the reward model does (rates answers) and *how* it's trained (the Bradley–Terry loss).
Here is what it physically is.

- ▶ Start from the SFT model π_{ref} , the same transformer backbone.
- ▶ A **head** is the small output layer stacked on top of that backbone. The **LM head** turns the backbone's features into a probability over the next token.
- ▶ Swap it for a **scalar head**: a layer that emits *one number*, $r_\phi(x, y)$, for the whole (prompt, response). Train it with the Bradley–Terry loss from the last slide.
- ▶ So the reward model is “*the same network, one number out instead of a token distribution.*”

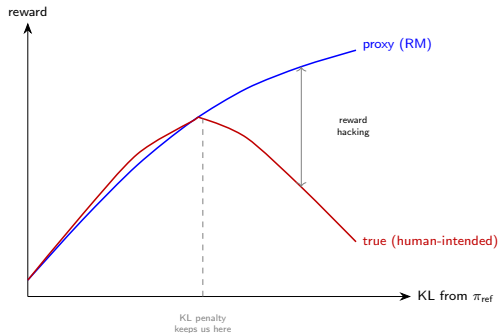


- ▶ We now have a reward model r_ϕ that scores answers. **The RL loop:** the policy generates a response, r_ϕ scores it, and we update the policy's weights to earn higher scores, with PPO or GRPO.
- ▶ The policy starts as π_{ref} (the frozen SFT model) and improves from there.
- ▶ *One catch:* r_ϕ is only a *learned proxy* for human taste, not the real thing. Optimize it too hard and the model learns to *game* it. The next slide shows how that happens, and the fix.

Why the KL Penalty Exists: Reward Over-Optimization

The RL objective is the reward-model score minus a leash to the reference policy (β sets how hard we pull back toward π_{ref}):

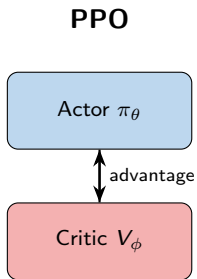
$$r_{\text{total}} = r_{\phi} - \beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}})$$



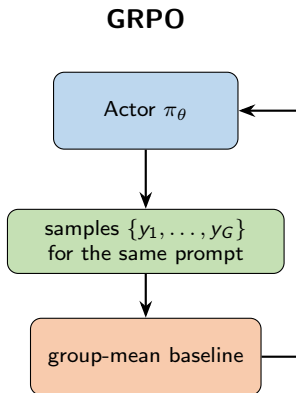
The reward model is a **proxy**. Optimize it too hard and you climb the proxy while the *real* objective collapses, *Goodhart's law*.

- ▶ Proxy reward (RM) keeps rising.
- ▶ True reward rises, then *falls*.
- ▶ **Why the leash works:** the penalty grows with KL distance, so each step away from π_{ref} costs more. Past the peak, the extra proxy gain is outweighed by the penalty, so the best-scoring policy sits *near the peak*, not in the hacked region.

The RL Back-End: PPO → GRPO



the fragile, expensive
half of PPO



$$\hat{A}_i = \frac{r_i - \text{mean}(r)}{\text{std}(r)}$$

no critic

The critic estimates a response's expected reward to form the advantage: PPO's brittle, expensive component.

GRPO deletes it: the baseline is just the average reward over a group of samples for the same prompt. PPO was the original RLHF back-end; **GRPO is now common for LLM RL.**

Shao et al. **DeepSeekMath** (GRPO), 2024; DeepSeek-AI, **DeepSeek-R1**, 2025.

- ▶ **The pain.** The full pipeline is heavy: Stage 2 trains a *separate* reward model, and Stage 3 runs an RL loop juggling several models at once (policy, reference, reward model, and, for PPO, a critic), which is finicky to tune.
- ▶ **The question.** The reward model exists only to score the preferences we *already collected*. What if we skip it, and the RL loop, and tune the policy *straight from the preference pairs* (y^+, y^-)?
- ▶ **DPO (Rafailov et al., 2023):** exactly that, a single supervised-style loss on preference pairs, applied directly to **the policy** π_θ (the SFT model we are fine-tuning). *No reward model. No RL loop.*

RLHF: preferences $\rightarrow$ train reward model $\rightarrow$ RL loop $\rightarrow$ policy

DPO: preferences $\rightarrow$ one loss $\rightarrow$ policy

But how can you train toward a reward you never built? The plain-English idea is on the next slide, then the loss it leads to. The derivation, a page of algebra starting from the RLHF objective, is in Rafailov et al. (2023).

Delete the judge: the chatbot is already its own judge.

- ▶ Look at how much **more likely** the current chatbot is to give an answer than the *original* (starting) model was. If it has become much more likely to say answer A, that *is* the chatbot “rating A highly.” *That ratio, “how much more do I favour this than I used to”, is the score.*
- ▶ So DPO throws away the separate judge *and* the practice loop, and trains the chatbot directly so that, for each comparison, **its own favouring of the preferred answer beats its favouring of the rejected one**. One model, one training step, no loop.
- ▶ Rafailov et al. make this exact. The next slide shows the loss, and why their paper is titled *“your LM is secretly a reward model.”*

The Payoff: Your LM Is Secretly a Reward Model

Put the DPO loss next to the *reward-model* loss from Stage 2. They are the **same shape**:

$$L_{\text{RM}} = -\log \sigma(r_{\phi}(y^+) - r_{\phi}(y^-))$$

$$L_{\text{DPO}} = -\log \sigma(\hat{r}(y^+) - \hat{r}(y^-)), \quad \hat{r}(y) = \beta \log \frac{\pi_{\theta}(y)}{\pi_{\text{ref}}(y)}$$

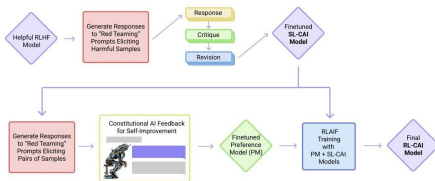
- ▶ The only change: the learned reward r_{ϕ} is replaced by the *implicit* reward $\hat{r} = \beta \log \pi_{\theta} / \pi_{\text{ref}}$, read straight off the policy. *So the policy is the reward model*; that is why “your LM is secretly a reward model.”
- ▶ **Honesty caveat:** the equivalence is exact only if the reward model were Bradley–Terry-optimal. In practice **DPO** $\neq$ **PPO**: DPO gives up the RL loop’s fresh on-policy samples.

The same Bradley–Terry loss, with the reward read off the policy itself: no reward model and no RL loop. *That’s the whole trick.*

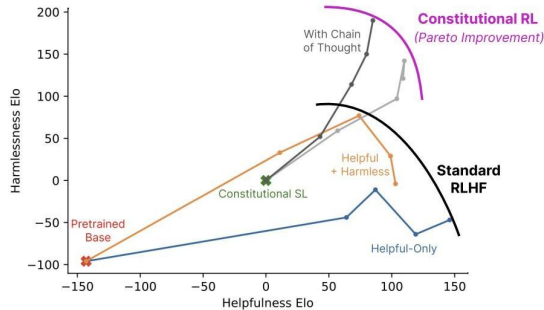
DPO's simplicity has a cost: you lose the control knobs the RL loop gave you.

- ▶ **It constrains only the gap, nothing else.** The loss just pushes the preferred answer's favoring above the rejected one's. With no on-policy sampling to pin down absolute behaviour, it can push *both* down, leaking probability onto unrelated answers it never saw. RLHF's RL loop and "don't-drift" anchor would catch that; DPO has neither.
- ▶ **Length bias (shared, but harder to patch).** If the data favours long answers, *both* RLHF and DPO turn verbose. RLHF lets you subtract an explicit length penalty from the reward; DPO has no separate reward to adjust, so length control must be bolted on as extra loss terms.
- ▶ **No fresh samples.** The policy only ever sees the fixed preference pairs, so it can drift off that distribution, the price of dropping the RL loop.

- **Motivation:** as models approach or exceed human performance, it gets harder for humans to label every output. *Move the labeller from human to AI; the only human input is the constitution*, a short list of written principles the model uses to critique and revise its own answers.
- **Constitutional AI** has two phases:
 - **Supervised:** sample $\rightarrow$ self-critique $\rightarrow$ revise $\rightarrow$ fine-tune on revisions.
 - **RL:** AI labels which response is better $\rightarrow$ train a reward model (RM) $\rightarrow$ RL-finetune with it (just like RLHF).

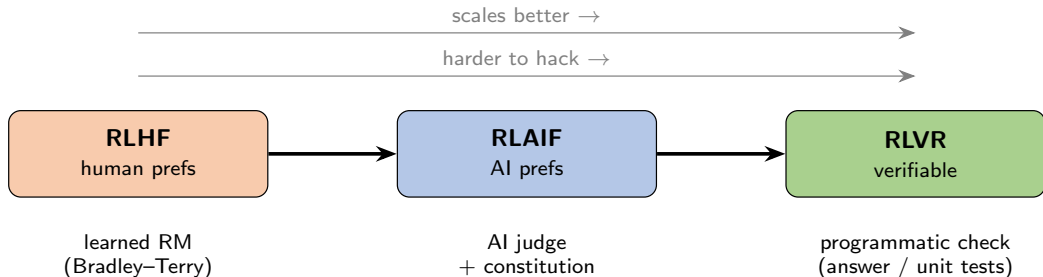


- ▶ Fine-tuning with **AI-generated feedback** can match or exceed models fine-tuned with human feedback, on *both* helpfulness and harmlessness.
- ▶ The plot rates models by **Elo**, a head-to-head win-rate score borrowed from chess (higher = preferred more often in pairwise comparisons).



The Climax: RL with Verifiable Rewards (RLVR)

- ▶ When the answer is **checkable** (math is right/wrong, code passes tests), the reward needs *no learned model* and **can't be hacked** the way an RM can.
- ▶ This is how modern **reasoning models** are trained: long **chain-of-thought** (the model writes out its reasoning steps before answering), optimized with **GRPO** against a programmatic check.



One RLVR step (no reward model anywhere):

1. the policy samples several answers to a *checkable* question;
2. a *rule* grades each (right answer? right format?), giving reward 1 or 0;
3. GRPO uses those rewards to update the policy. *Nothing to hack: the checker is exact.*

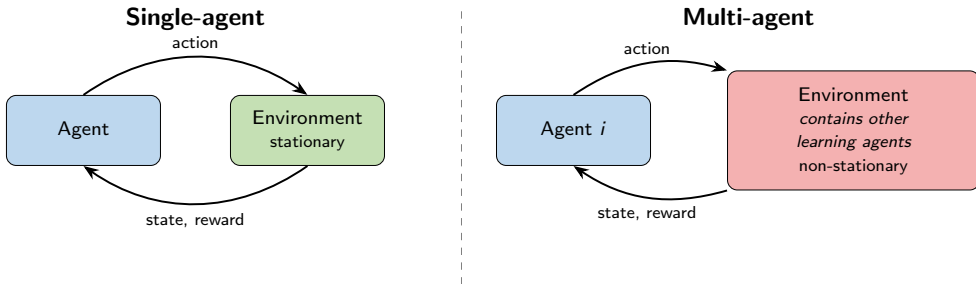
DeepSeek-R1-Zero ran exactly this on a *base* model, **no SFT at all**, with reward = “answer correct” + “reasoning in <think> tags”:

- ▶ *Reasoning emerged on its own*: longer chains of thought, self-checking, backtracking, even an “*aha moment*” where it stops and corrects itself. Nobody taught it to.
- ▶ AIME 2024 math: **15.6% → 71.0%** (86.7% with majority vote), matching OpenAI o1.
- ▶ Catch: raw R1-Zero was hard to read (language-mixing), so the full **R1** adds a small SFT “cold start.” But R1-Zero proved the point: *reasoning can be grown by pure RL from verifiable rewards.*

Thread 2 of 3

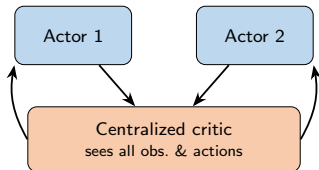
Multi-Agent RL

The RLHF deep dive is done. A shorter overview now: what changes when other learning agents share your environment.

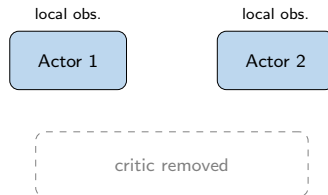


- ▶ Each agent's environment is **non-stationary**: the others are updating their policies at the same time, so the ground shifts as they learn; single-agent convergence guarantees break.
- ▶ Formalized as a **Markov game** (stochastic game), the multi-agent generalization of the MDP.
- ▶ Three regimes along one axis: **cooperative** / **competitive** / **mixed**.

Training (centralized)

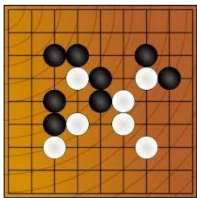


Execution (decentralized)



- **At training:** a centralized critic sees every agent's observations and actions. Because it accounts for what the others did, each agent's learning signal becomes *stationary*, defusing the non-stationarity problem from the last slide.
- **At execution:** each agent keeps only its *own actor* acting on its *local observation*; the critic is gone.
- This is the dominant MARL paradigm, the key idea to remember.

- ▶ An agent improves by **playing against past versions of itself**: *policy* $\rightarrow$ *plays past versions* $\rightarrow$ *game data* $\rightarrow$ *improve* $\rightarrow$ (repeat).
- ▶ This creates an **automatic curriculum**: the opponent gets harder exactly as you do, with no hand-designed difficulty schedule.



AlphaGo / AlphaZero

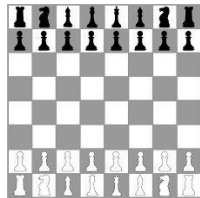
(Silver et al.)

AlphaStar
StarCraft II

(Vinyals et al., 2019)

OpenAI Five

Dota 2 (2019)



Chess (AlphaZero)

Seen in Part I: the hide-and-seek agents learned to use tools the same way, from an automatic curriculum of ever-better opponents.

Thread 3 of 3

Robotics

What breaks when the environment is physical: sample cost, safety, resets, and the sim-to-real gap.

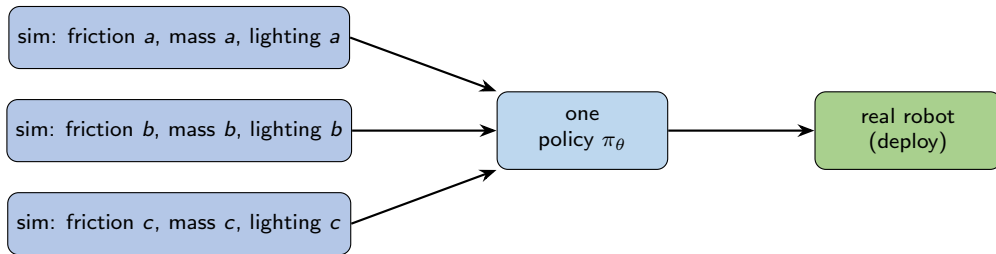
Simulation

- ▶ cheap; millions of steps
- ▶ parallelizable across machines
- ▶ failures are free
- ▶ *but*: a **reality gap**

Real hardware

- ▶ expensive, slow, safety-critical
- ▶ **resets are a real problem** (who picks the object back up?)
- ▶ sparse / hard-to-engineer reward

Four obstacles (**sample efficiency, safety, resets, reward specification**) are why off-the-shelf RL does not transfer to hardware.



Train across a *distribution* of randomized sims (friction, mass, latency, textures, lighting) so the real world looks like just one more sample.

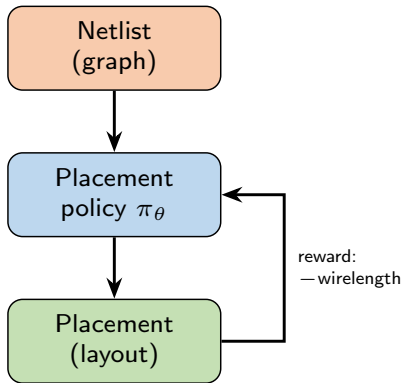
Headline successes: OpenAI Dactyl (in-hand cube, Akkaya et al. 2019); legged locomotion (ANYmal, Hwangbo et al. 2019; massively parallel sim, Rudin et al. 2022).

Domain randomization: Tobin et al. [Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World](#), 2017.

Illustrative example: chip floorplanning (Mirhoseini et al., *Nature* 2021; later named **AlphaChip**):

- ▶ Place macros on a chip canvas to minimize wirelength, congestion, and density.
- ▶ The striking idea: *pre-train* a placement policy on many netlists, then place a *new* chip in a forward pass, instead of optimizing each one from scratch.

Also deployed for: datacenter resource scheduling, compiler / device placement, real-time ad bidding.



The transferable idea: all are *combinatorial optimization as sequential decision-making*: one MDP framing with an *engineered* reward (wirelength, runtime, ROI). The application details are not the lesson.

- ▶ **The language.** An MDP $(\mathcal{S}, \mathcal{A}, R, P, \gamma)$; a policy maps states to actions; the goal is the largest expected discounted return.
- ▶ **Value-based RL.** Nudge $Q(s, a)$ toward the Bellman target $r + \gamma \max_{a'} Q(s', a')$ and act greedily, with some ε -exploration. Q-learning needs no model of the environment.
- ▶ **Policy-based RL.** Step along $\sum_t \hat{A}_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$: make better-than-average actions more likely. Baselines and advantages cut the variance; PPO clips each step; GRPO takes its baseline from a group of attempts.
- ▶ **The real world.** Often the reward is the hard part. RLHF learns it from comparisons and needs a KL leash against reward hacking; verifiable rewards remove the learned judge; robots add safety and the sim-to-real gap.

RLHF.

- ▶ Bradley & Terry. Rank Analysis of Incomplete Block Designs. *Biometrika*, 1952. Christiano et al. Deep RL from Human Preferences. NeurIPS 2017.
- ▶ Stiennon et al. Learning to Summarize from Human Feedback. 2020.
- ▶ Ouyang et al. Training Language Models to Follow Instructions with Human Feedback (InstructGPT). 2022.
- ▶ Bai et al. Training a Helpful and Harmless Assistant with RLHF. 2022.
- ▶ Gao, Schulman, Hilton. Scaling Laws for Reward Model Overoptimization. ICML 2023.

Modern alignment.

- ▶ Rafailov, Sharma, Mitchell, Ermon, Manning, Finn. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. NeurIPS 2023.
- ▶ Xu et al. Is DPO Superior to PPO for LLM Alignment? ICML 2024.
- ▶ Shao et al. DeepSeekMath (GRPO). 2024.
- ▶ Bai et al. Constitutional AI: Harmlessness from AI Feedback. 2022.

- ▶ DeepSeek-AI. DeepSeek-R1 / R1-Zero. *Nature* 645:633–638, 2025 (arXiv 2501.12948). Lambert et al. Tülu 3 (RLVR). 2024.

Multi-Agent RL.

- ▶ Lowe et al. MADDPG. 2017. Sunehag et al. VDN. 2018. Rashid et al. QMIX. 2018.
- ▶ Vinyals et al. AlphaStar (StarCraft II). 2019. OpenAI. Dota 2 with Large Scale Deep RL (OpenAI Five). 2019.

Robotics.

- ▶ Tobin et al. Domain Randomization. 2017. Akkaya et al. Solving Rubik's Cube with a Robot Hand (Dactyl). 2019.
- ▶ Hwangbo et al. Learning Agile and Dynamic Motor Skills for Legged Robots (ANYmal). 2019. Rudin et al. Massively Parallel Sim. 2022.
- ▶ Luo et al. SERL. 2024.

Systems.

- ▶ Mirhoseini et al. A Graph Placement Methodology for Fast Chip Design. *Nature*, 2021 (later named **AlphaChip**; Addendum, *Nature* 634:E10, 2024).

These slides have been adapted from Anna Goldie, *RL in the Real World: From Chip Design to LLMs*, guest lecture in **CS224R: Deep Reinforcement Learning**, Stanford.

- [1] Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8, 229–256.
- [2] Schulman, J., Levine, S., Moritz, P., Jordan, M. I., & Abbeel, P. (2015). Trust Region Policy Optimization. *ICML*.
- [3] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization Algorithms. *arXiv:1707.06347*.
- [4] Shao, Z., Wang, P., Zhu, Q., et al. (2024). DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models (GRPO). *arXiv:2402.03300*.